BÀI TOÁN K-ARMED BANDIT TRONG REINFORCEMENT LEARNING

1 Giới thiệu bài toán k-armed bandit

Đặc điểm quan trọng nhất phân biệt *Reinforcement Learning* (Học tăng cường) với các loại hình học máy khác là nó sử dụng thông tin huấn luyện để *đánh giá* (evaluate) các hành động đã thực hiện, thay vì *hướng dẫn* (instruct) hành động nào là đúng. Phản hồi đánh giá (evaluative feedback) chỉ cho biết hành động được thực hiện tốt đến mức nào, chứ không cho biết đó có phải là hành động tốt nhất hay tồi tệ nhất có thể hay không. Điều này tạo ra nhu cầu cần phải chủ động *khám phá* (active exploration) để tìm kiếm các hành vi tốt.

Bài toán **k-armed bandit** (trong tài liệu gốc sử dụng ký hiệu *n-armed bandit*) là một phiên bản đơn giản hóa, mang tính chất phi liên kết (nonassociative) của Reinforcement Learning. Đây là bài toán cơ sở giúp chúng ta hiểu rõ nhất cách thức hoạt động của phản hồi đánh giá.

Định nghĩa bài toán: Hãy tưởng tượng bạn phải đối mặt liên tục với việc lựa chọn một trong k tùy chọn (hoặc hành động) khác nhau. Sau mỗi lần lựa chọn, bạn nhận được một phần thưởng (reward) bằng số, được rút ra từ một phân phối xác suất tĩnh (stationary probability distribution) phụ thuộc vào hành động bạn vừa chọn. Mục tiêu của bạn là tối đa hóa tổng phần thưởng kỳ vọng (expected total reward) trong một khoảng thời gian dài, ví dụ như qua 1000 lần chọn hành động (hay còn gọi là *time steps*).

Nguồn gốc tên gọi: Tên gọi *k-armed bandit* bắt nguồn từ sự tương đồng với máy đánh bạc (slot machine) hay còn gọi là "máy cướp một tay" (one-armed bandit). Tuy nhiên, bài toán của chúng ta có k cần gạt thay vì chỉ một. Mỗi lần chọn hành động tương đương với việc kéo một cần gạt, và phần thưởng là số tiền trúng giải. Bằng cách thử lặp đi lặp lại, mục tiêu là tối đa hóa tiền thắng cược bằng cách tập trung kéo những cần gạt mang lại lợi ích cao nhất.

2 Các khái niệm, định nghĩa và ký hiệu trong thiết lập bài toán

Để giải quyết bài toán toán học này, chúng ta cần định nghĩa các ký hiệu chuẩn xác như sau:

- t: Bước thời gian (time step), $t = 1, 2, 3, \ldots$
- A_t: Hành động (Action) được lựa chọn tại bước thời gian t.

- R_t: Phần thưởng (Reward) nhận được tại bước thời gian t, sau khi thực hiện hành động A_t.

Trong bài toán k-armed bandit, mỗi hành động a có một phần thưởng trung bình hoặc kỳ vọng (expected or mean reward) khi hành động đó được chọn. Ta gọi đây là **giá trị (value)** của hành động đó, ký hiệu là $q(a)$.

Về mặt toán học, $q(a)$ chính là **giá trị kỳ vọng (Expected Value)** của phần thưởng R_t khi biết hành động được chọn là a. Giá trị kỳ vọng trong thống kê là giá trị trung bình theo trọng số xác suất của tất cả các kết quả có thể xảy ra. Công thức được viết là:

$$q(a) \doteq \mathbb{E}[R_t \mid A_t = a] \tag{1}$$

Nếu bạn biết chính xác giá trị $q(a)$ của từng hành động, bài toán sẽ trở nên vô cùng tầm thường: bạn chỉ việc luôn chọn hành động có giá trị cao nhất. Tuy nhiên, trong thực tế, chúng ta không biết chắc chắn các giá trị này. Ta chỉ có các **giá trị ước lượng (estimated value)** của hành động a tại bước thời gian t, được ký hiệu là $Q_t(a)$. Mục tiêu là làm cho $Q_t(a)$ càng gần với $q(a)$ càng tốt.

Dựa trên các ước lượng này, tại bất kỳ bước thời gian nào, luôn có ít nhất một hành động có giá trị ước lượng lớn nhất. Đây được gọi là hành động **greedy** (tham lam).

- **Khai thác (Exploitation)**: Khi bạn chọn hành động greedy, bạn đang khai thác kiến thức hiện tại của mình để tối đa hóa phần thưởng tức thời.
- **Khám phá (Exploration)**: Khi bạn chọn một trong các hành động phi tham lam (nongreedy), bạn đang khám phá để cải thiện ước lượng về giá trị của các hành động đó trong tương lai.

Sự xung đột (conflict) giữa Exploration và Exploitation là một thách thức cốt lõi đặc trưng của Reinforcement Learning. Khai thác mang lại phần thưởng ngắn hạn, nhưng Khám phá lại mang về phần thưởng tổng thể lớn hơn trong dài hạn.

3 Phương pháp Action-value, Greedy và ϵ-greedy

3.1 Phương pháp Action-value và Trung bình mẫu

Các phương pháp **Action-value** là các phương pháp ước lượng giá trị của hành động và sử dụng các ước lượng đó để đưa ra quyết định lựa chọn hành động.

Cách tự nhiên nhất để ước lượng giá trị thật $q(a)$ là tính trung bình các phần thưởng thực tế nhận được mỗi khi hành động a được chọn. Cụ thể, nếu tại bước thời gian t, hành động a đã được chọn $N_t(a)$ lần, tạo ra các phần thưởng $R_1, R_2, \ldots, R_{N_t(a)}$,

thì giá trị ước lượng của nó là:

$$Q_t(a) \doteq \frac{R_1 + R_2 + \cdots + R_{N_t(a)}}{N_t(a)} \tag{2}$$

Nếu $N_t(a) = 0$, ta có thể gán cho $Q_t(a)$ một giá trị mặc định (ví dụ: $Q_1(a) = 0$). Theo *Luật số lớn (Law of large numbers)* trong xác suất thống kê, khi số lần chọn $N_t(a) \to \infty$, giá trị trung bình mẫu $Q_t(a)$ sẽ hội tụ về giá trị kỳ vọng thực tế $q(a)$. Phương pháp này được gọi là phương pháp **trung bình mẫu (sample-average)**.

3.2 Phương pháp Greedy

Quy tắc lựa chọn hành động đơn giản nhất là luôn chọn hành động có ước lượng Action-value cao nhất tại thời điểm hiện tại. Nếu có nhiều hành động có cùng giá trị lớn nhất, ta có thể chọn ngẫu nhiên một trong số chúng. Cú pháp toán học của hành động tham lam A_t^* được viết là:

$$A_t \doteq \arg\max_a Q_t(a) \tag{3}$$

Trong đó, toán tử $\arg\max_a$ biểu thị giá trị của biến a làm cho biểu thức theo sau đạt cực đại.

Phương pháp **greedy** luôn luôn *khai thác* (exploit) kiến thức hiện tại để tối đa hóa phần thưởng tức thì; nó không dành thời gian để thử nghiệm các hành động trông có vẻ kém hơn để xem liệu chúng có thực sự tốt hơn hay không.

3.3 Phương pháp ϵ-greedy

Một giải pháp thay thế đơn giản là hành xử greedy hầu hết mọi lúc, nhưng thi thoảng, với một xác suất nhỏ ϵ (epsilon), ta thay vì chọn greedy lại chọn ngẫu nhiên một cách độc lập từ tất cả các hành động với xác suất bằng nhau (bao gồm cả việc có thể vô tình chọn lại hành động greedy). Phương pháp sử dụng quy tắc chọn lựa gần như tham lam này được gọi là **ϵ-greedy**.

Lợi thế của phương pháp ϵ-greedy là khi số bước tiến tới vô cùng, mọi hành động sẽ được thử vô số lần (nghĩa là $N_t(a) \to \infty$ cho mọi a), đảm bảo rằng tất cả các $Q_t(a)$ sẽ hội tụ chính xác về $q(a)$.

3.4 Phân tích ví dụ 10-armed Testbed

Để so sánh hiệu quả của greedy và ϵ-greedy, tài liệu đề xuất bài thử nghiệm **10-armed testbed**. Trong bài thử nghiệm này, có 2000 bài toán bandit được tạo ngẫu nhiên với $k = 10$ hành động.

- **Phân phối xác suất**: Đối với mỗi bài toán, giá trị thực $q(a)$ được lấy mẫu từ một phân phối chuẩn (Gaussian distribution) với giá trị trung bình là 0 và phương sai là 1, ký hiệu là $\mathcal{N}(0, 1)$.

- Phần thưởng R_t tại bước t cũng được cộng thêm một nhiễu ngẫu nhiên phân phối chuẩn với trung bình là 0 và phương sai là 1, tức là $R_t \sim \mathcal{N}(q(A_t), 1)$.

Kết quả:

- **Phương pháp Greedy ($\epsilon = 0$):** Tăng trưởng phần thưởng rất nhanh ở những bước đầu, nhưng sau đó đi ngang và dừng lại ở mức thấp. Lý do là phương pháp greedy thường xuyên bị kẹt ở các hành động tối ưu cục bộ (suboptimal actions). Nó chỉ tìm được hành động tối ưu trong khoảng 30% số lần chạy.

- **Phương pháp ϵ-greedy với $\epsilon = 0.1$:** Khám phá nhiều hơn nên nhanh chóng tìm ra hành động tối ưu. Tuy nhiên, vì nó luôn chọn ngẫu nhiên 10% số lần, hiệu suất bị giới hạn không vượt quá 91% lựa chọn tối ưu.

- **Phương pháp ϵ-greedy với $\epsilon = 0.01$:** Cải thiện chậm hơn so với $\epsilon = 0.1$ ở giai đoạn đầu do tần suất khám phá ít hơn. Tuy nhiên, về dài hạn, phương pháp này sẽ vượt qua $\epsilon = 0.1$ và đạt đỉnh hiệu năng cao hơn vì xác suất chọn sai chỉ còn 1%.

3.5 Triển khai tăng dần (Incremental Implementation)

Để tính toán $Q_t(a)$ theo phương pháp trung bình mẫu (Công thức 2), nếu giữ nguyên toàn bộ lịch sử phần thưởng, yêu cầu bộ nhớ và tính toán sẽ tăng tiến tính theo thời gian. Ta có thể giải quyết việc này bằng công thức **cập nhật tăng dần** (chỉ cần tính toán $O(1)$ về bộ nhớ).

Giả sử ta chỉ xét một hành động duy nhất. Gọi Q_n là giá trị ước lượng sau $n-1$ lần hành động đó được chọn (nghĩa là trung bình của $n-1$ phần thưởng đầu tiên), và R_n là phần thưởng nhận được ở lần chọn thứ n. Trung bình của n phần thưởng được tính như sau:

$$\begin{aligned} Q_{n+1} &= \frac{1}{n}\sum_{i=1}^{n} R_i \\ &= \frac{1}{n}\left(R_n + \sum_{i=1}^{n-1} R_i\right) \\ &= \frac{1}{n}\left(R_n + (n-1)Q_n + Q_n - Q_n\right) \\ &= \frac{1}{n}\left(R_n + nQ_n - Q_n\right) \\ &= Q_n + \frac{1}{n}\Big[R_n - Q_n\Big] \end{aligned} \tag{4}$$

(Lưu ý: Trong tài liệu gốc ký hiệu là k thay vì n cho số thứ tự lần thử. Ở đây dùng n để tránh nhầm lẫn với k trong k-armed bandit).

Quy tắc cập nhật này có dạng tổng quát vô cùng quan trọng trong suốt lĩnh vực Reinforcement Learning:

$$NewEstimate \leftarrow OldEstimate + StepSize\Big[Target - OldEstimate\Big] \tag{5}$$

Trong đó biểu thức $[Target - OldEstimate]$ chính là **sai số (error)** của ước lượng. Ước lượng cũ sẽ dịch chuyển một bước (*StepSize*, ở đây là $\frac{1}{n}$) hướng về phía giá trị mục tiêu (*Target*, ở đây là R_n).

4 Exercise 2.1

Đề bài: Trong sự so sánh được thể hiện ở Hình 2.1 (Figure 2.1), phương pháp nào sẽ hoạt động tốt nhất trong dài hạn (in the long run) xét về phần thưởng tích lũy (cumulative reward) và xác suất tích lũy của việc chọn hành động tốt nhất (cumulative probability of selecting the best action)? Nó sẽ tốt hơn bao nhiêu? Hãy biểu diễn câu trả lời của bạn một cách định lượng (quantitatively).

Lời giải chi tiết:

Trong dài hạn (khi $t \to \infty$), phương pháp **ϵ-greedy với $\epsilon = 0.01$** sẽ là phương pháp hoạt động tốt nhất.

Để giải thích định lượng cho điều này, chúng ta cần tính xác suất để mỗi phương pháp chọn được hành động tối ưu (optimal action) sau khi nó đã có đủ thời gian để khám phá và tìm ra chính xác đâu là hành động tốt nhất.

Trong thuật toán ϵ-greedy, ở bất kỳ bước thời gian nào, tác tử (agent) sẽ:

- Chọn hành động *greedy* (hành động tốt nhất theo đánh giá hiện tại) với xác suất là $1 - \epsilon$.
- Khám phá (*explore*) bằng cách chọn ngẫu nhiên một trong tất cả k hành động (bao gồm cả việc có thể vô tình chọn lại hành động tốt nhất) với xác suất ϵ. Do đó, xác suất vô tình chọn trúng hành động tốt nhất trong lúc khám phá là $\frac{\epsilon}{k}$.

Bài toán *10-armed testbed* có $k = 10$. Xác suất tiệm cận (asymptotic probability) để chọn hành động tốt nhất trong dài hạn, ký hiệu là $P(\text{optimal})$, được tính bằng công thức:

$$P(\text{optimal}) = (1 - \epsilon) + \frac{\epsilon}{k} \tag{6}$$

1. Phân tích về xác suất chọn hành động tốt nhất:

- **Với phương pháp $\epsilon = 0.1$:**

$$P_{0.1}(\text{optimal}) = (1 - 0.1) + \frac{0.1}{10} = 0.9 + 0.01 = 0.91 \text{ (hay 91\%)} \tag{7}$$

- **Với phương pháp $\epsilon = 0.01$:**

$$P_{0.01}(\text{optimal}) = (1 - 0.01) + \frac{0.01}{10} = 0.99 + 0.001 = 0.991 \text{ (hay 99.1\%)} \quad (8)$$

Kết luận về xác suất: Trong dài hạn, phương pháp $\epsilon = 0.01$ sẽ chọn hành động tốt nhất với tần suất cao hơn phương pháp $\epsilon = 0.1$ là:

$$0.991 - 0.91 = 0.081 \text{ (tức là cao hơn chính xác 8.1\%)} \quad (9)$$

2. Cumulative Reward:

Bởi vì phương pháp $\epsilon = 0.01$ chọn hành động tối ưu thường xuyên hơn **8.1%** so với phương pháp $\epsilon = 0.1$, phần thưởng trung bình mỗi bước (average reward per step) của nó trong dài hạn cũng sẽ cao hơn tương ứng.

Cụ thể, gọi q^* là giá trị kỳ vọng của hành động tốt nhất. Với sự chênh lệch 8.1% trong xác suất chọn q^*, tốc độ gia tăng của phần thưởng tích lũy (độ dốc của đường cumulative reward) của phương pháp $\epsilon = 0.01$ sẽ dốc hơn. Theo thời gian $t \to \infty$, khoảng cách về tổng phần thưởng tích lũy giữa $\epsilon = 0.01$ và $\epsilon = 0.1$ sẽ ngày càng nới rộng ra vô hạn (tốt hơn $8.1\% \times q^*$ cho mỗi bước thời gian).

*(Lưu ý: Phương pháp *greedy* $\epsilon = 0$ là tệ nhất trong dài hạn vì nó bị mắc kẹt ở các hành động *suboptimal* trong khoảng 2/3 số trường hợp và ngừng hoàn toàn việc cải thiện).*

5 Thiết lập đánh giá "10-armed Testbed"

Để so sánh hiệu quả của các phương pháp *Greedy* và *ϵ-greedy*, giáo trình của Sutton & Barto sử dụng một bộ khung đánh giá tiêu chuẩn gọi là **10-armed Testbed**. Đây là một môi trường mô phỏng giúp chúng ta quan sát hành vi của thuật toán trong điều kiện có nhiễu.

5.1 Cấu trúc của bộ thử nghiệm

Thiết lập này bao gồm 2000 bài toán k-armed bandit độc lập với $k = 10$. Đối với mỗi bài toán:

- **Giá trị thực ($q_*(a)$):** Đối với mỗi hành động $a \in \{1, \ldots, 10\}$, giá trị thực được chọn ngẫu nhiên từ một **Phân phối chuẩn (Gaussian Distribution)** với trung bình bằng 0 và phương sai bằng 1:

$$q_*(a) \sim \mathcal{N}(0, 1) \quad (10)$$

- **Phần thưởng (R_t):** Khi tác tử chọn hành động A_t tại bước thời gian t, phần thưởng thực tế R_t sẽ được lấy mẫu từ một phân phối chuẩn có trung bình là

$q_*(A_t)$ và phương sai là 1:

$$R_t \sim \mathcal{N}(q_*(A_t), 1) \tag{11}$$

- **Quy mô:** Mỗi phương pháp được chạy trong 1000 bước thời gian (time steps) và được lặp lại qua 2000 phiên bản bài toán khác nhau để lấy giá trị trung bình.

5.2 Tại sao sử dụng thiết lập này để đánh giá?

Thiết lập 10-armed Testbed không phải là một bài toán ngẫu nhiên đơn thuần, nó mang những ý nghĩa thống kê quan trọng để đánh giá học máy:

1. **Tính khách quan thông qua Trung bình mẫu (Statistical Significance):** Nếu chỉ chạy một bài toán duy nhất, kết quả có thể do may mắn (ví dụ: tác tử chọn trúng máy tốt nhất ngay từ đầu). Bằng cách lấy trung bình trên 2000 bài toán, các yếu tố nhiễu ngẫu nhiên bị triệt tiêu, giúp ta thấy được hiệu suất thực sự của thuật toán.

2. **Đánh giá khả năng Exploration vs Exploitation:** Vì phần thưởng R_t có độ lệch chuẩn là 1, nên phần thưởng nhận được từ hành động tốt nhất và hành động kém hơn có thể bị chồng lấn lên nhau. Điều này bắt buộc thuật toán phải có cơ chế **khám phá** đủ mạnh để không bị đánh lừa bởi những phần thưởng cao ngẫu nhiên từ các máy tồi.

3. **Khả năng hội tụ:** Thiết lập này cho phép quan sát tốc độ hội tụ. Một phương pháp tốt không chỉ cần tìm ra máy tốt nhất mà còn phải tìm ra nó thật nhanh để tối đa hóa phần thưởng tích lũy.

4. **Đo lường xác suất hành động tối ưu (Optimal Action Probability):** Vì chúng ta là người thiết lập môi trường, chúng ta biết chính xác máy nào là tốt nhất (q_* nào lớn nhất). Từ đó, ta có thể vẽ biểu đồ tỉ lệ phần trăm số lần tác tử chọn đúng máy tốt nhất theo thời gian — đây là thước đo chuẩn xác nhất cho sự thông minh của thuật toán.

5.3 Phân tích kết quả thực nghiệm

Dựa trên kết quả từ thiết lập này (Hình 2.1 trong tài liệu):

- Phương pháp **Greedy** nhanh chóng bị bão hòa vì nó thường xuyên bị kẹt vào một hành động có vẻ tốt ở giai đoạn đầu nhưng thực tế lại không phải là tối ưu.

- Phương pháp ϵ**-greedy** tiếp tục cải thiện vì nó không bao giờ ngừng khám phá. Ngay cả khi ϵ nhỏ, nó vẫn đảm bảo rằng về mặt lý thuyết, xác suất tìm thấy máy tối ưu sẽ tiến dần đến 100% khi $t \to \infty$.

Ký hiệu và thuật ngữ

Dưới đây là bảng tổng hợp các ký hiệu toán học và thuật ngữ chuyên ngành được sử dụng trong bài viết này:

Ký hiệu	**Ý nghĩa**
k	Số lượng hành động (số "tay" của máy đánh bạc).
t	Bước thời gian (Time step).
A_t	Hành động được lựa chọn tại thời điểm t.
R_t	Phần thưởng nhận được sau khi thực hiện hành động A_t.
$q(a)$	Giá trị thực (giá trị kỳ vọng) của hành động a.
$Q_t(a)$	Giá trị ước lượng của hành động a tại thời điểm t.
$N_t(a)$	Số lần hành động a đã được chọn tính đến trước thời điểm t.
$\mathbb{E}[X]$	Giá trị kỳ vọng của biến ngẫu nhiên X.
$\doteq$	Ký hiệu định nghĩa (Bằng theo định nghĩa).
ϵ	Tham số xác suất dành cho việc khám phá (Exploration).
$\arg\max_a$	Giá trị của a làm cho biểu thức đi kèm đạt cực đại.
$\mathcal{N}(\mu, \sigma^2)$	Phân phối chuẩn (Gaussian) với trung bình μ và phương sai σ^2.
α (hoặc $1/n$)	Tham số kích thước bước (Step-size) trong cập nhật tăng dần.
Exploitation	Khai thác: Chọn hành động tốt nhất hiện tại.
Exploration	Khám phá: Thử các hành động mới để thu thập thông tin.
Stationary	Tĩnh: Phân phối phần thưởng không thay đổi theo thời gian.

Ghi chú: Các ký hiệu được tuân thủ theo tiêu chuẩn của giáo trình Reinforcement Learning: An Introduction (Sutton & Barto).

Vi Phân Tự Động Trong Học Máy (Automatic Differentiation in Machine Learning)

1 Sơ lược về Vi phân tự động

Vi phân tự động (Automatic Differentiation - AD) là một kỹ thuật tính toán đạo hàm của các hàm số được lập trình trên máy tính một cách chính xác đến mức sai số máy (machine precision). Khác với **Vi phân số học (Numerical Differentiation)** vốn dễ bị lỗi làm tròn (round-off error) và lỗi cắt cụt (truncation error), cũng như khác với **Vi phân ký hiệu (Symbolic Differentiation)** thường dẫn đến sự phình to biểu thức (expression swell) một cách phức tạp, AD hoạt động bằng cách áp dụng có hệ thống quy tắc xích (chain rule) của giải tích ở mức độ các phép toán cơ bản (cộng, trừ, nhân, chia, sin, cos, exp,...).

Ảnh hưởng của AD đã được thiết lập và sử dụng rộng rãi từ lâu trong các lĩnh vực yêu cầu tính toán khoa học phức tạp như động lực học chất lưu (computational fluid dynamics), kỹ thuật hạt nhân, khoa học khí quyển và thiên văn học. Trong lĩnh vực **Học máy (Machine Learning)**, dù AD từng bị cộng đồng lờ đi trong một thời gian dài, nó thực chất chính là nền tảng cốt lõi của thuật toán lan truyền ngược (Backpropagation) nổi tiếng. Ngày nay, AD là trái tim của các framework Deep Learning hiện đại (như TensorFlow, PyTorch), cho phép tối ưu hóa các mô hình khổng lồ mà không cần lập trình viên phải tự tính toán đạo hàm bằng tay.

2 Vi phân số học (Finite Differences) so với Vi phân tự động

Để thấy được giá trị cốt lõi của Vi phân tự động trong Học máy, chúng ta cần đặt nó lên bàn cân với phương pháp truyền thống và trực quan nhất: **Vi phân số học**, hay cụ thể hơn là phương pháp **Sai phân hữu hạn (Finite Differences)**.

1. Phương pháp Sai phân hữu hạn

Phương pháp này dựa trực tiếp vào định nghĩa giới hạn của đạo hàm. Đối với một hàm $f : \mathbb{R}^n \to \mathbb{R}$, đạo hàm riêng theo biến x_i được xấp xỉ bằng công thức sai phân tiến (forward difference):

$$\frac{\partial f(\mathbf{x})}{\partial x_i} \approx \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h} \tag{1}$$

Trong đó $\mathbf{e}_i$ là vector đơn vị thứ i, và h là một bước nhảy có giá trị cực nhỏ ($0 < h \ll 1$).

Phương pháp này coi hàm f như một "hộp đen"(chỉ quan tâm đầu vào và đầu ra), rất dễ lập trình nhưng lại mang những nhược điểm chí mạng:

- **Hiệu năng cực kém cho mô hình lớn:** Để tính được vector Gradient cho hàm có n biến, ta phải đánh giá hàm f tới $\mathcal{O}(n)$ lần. Nếu mạng neural có 1 triệu trọng số, ta phải chạy tiến mô hình 1 triệu lần chỉ để tính đạo hàm cho một bước học.
- **Sự bất ổn định số học (Ill-conditioned):** Phương pháp này đối mặt với sự đánh đổi không thể khắc phục giữa hai loại sai số. Nếu h lớn, ta bị *Sai số cắt cụt (Truncation Error)*. Nếu h quá nhỏ, ta bị *Sai số làm tròn (Round-off Error)* do máy tính gặp giới hạn bộ nhớ khi trừ hai số khổng lồ gần bằng nhau, làm mất đi các bit có ý nghĩa.

2. Sự vượt trội của Vi phân tự động (AD)

Thay vì xấp xỉ đạo hàm ở tầm nhìn vĩ mô như sai phân hữu hạn, AD đi sâu vào cấu trúc vi mô của đoạn code. Nó dựa trên nguyên lý: Mọi chương trình máy tính cuối cùng cũng chỉ là tổ hợp của một lượng hữu hạn các phép toán cơ bản (cộng, trừ, nhân, chia) và hàm cơ sở ($\sin, \cos, \exp, \log$).

Bằng cách xây dựng đồ thị tính toán và áp dụng chính xác quy tắc xích của giải tích lên từng mắt xích nhỏ này, AD mang lại sự đột phá:

- **Độ chính xác tuyệt đối:** Hoàn toàn miễn nhiễm với sai số cắt cụt và sai số làm tròn, đạt độ chính xác ở mức machine precision.
- **Thích ứng linh hoạt:** Có khả năng tính đạo hàm xuyên qua các cấu trúc điều khiển của ngôn ngữ lập trình như vòng lặp (`for`, `while`) hay lệnh rẽ nhánh (`if/else`).

3 Quy tắc xích cho hàm đa biến dưới dạng tổng của tích

Giả sử chúng ta có một hàm đa biến $f : \mathbb{R}^n \to \mathbb{R}^m$. Quá trình tính toán hàm này có thể được chia thành các biến trung gian. Xét một biến đầu ra y phụ thuộc vào các biến trung gian $v_1, v_2, \ldots, v_k$, và các biến trung gian này lại phụ thuộc vào các biến đầu vào $x_1, x_2, \ldots, x_n$.

Theo quy tắc xích cơ bản của giải tích đa biến, đạo hàm riêng của y theo một biến đầu vào x_j được biểu diễn dưới dạng tổng của tích các đạo hàm riêng dọc theo tất cả các đường dẫn nối x_j với y trong đồ thị tính toán:

$$\frac{\partial y}{\partial x_j} = \sum_{i=1}^{k} \frac{\partial y}{\partial v_i} \frac{\partial v_i}{\partial x_j} \tag{2}$$

Giải thích: Công thức này mang ý nghĩa rằng sự thay đổi của x_j sẽ ảnh hưởng đến tất cả các biến trung gian v_i mà nó tham gia định hình (đại diện bởi $\frac{\partial v_i}{\partial x_j}$). Sau đó, mỗi sự thay đổi của v_i lại tiếp tục lan truyền để tạo ra sự thay đổi ở đầu ra y (đại diện bởi $\frac{\partial y}{\partial v_i}$). Tổng tất cả các tác động gián tiếp này sẽ cho ta tác động toàn phần của x_j lên y.

Ví dụ minh họa

Bài toán: Cho hai biến đầu vào x_1, x_2. Ta định nghĩa các biến trung gian $v_1 = x_1^2 + x_2$ và $v_2 = x_1 x_2$. Hàm mục tiêu cuối cùng là $y = 3v_1 + v_2^2$. Hãy tính đạo hàm riêng $\frac{\partial y}{\partial x_1}$.

Forward Pass:

$$\begin{aligned} v_1 &= x_1^2 + x_2 \\ v_2 &= x_1 x_2 \\ y &= 3v_1 + v_2^2 \end{aligned}$$

Backward Pass:

$$\begin{aligned} \frac{\partial y}{\partial v_1} &= 3 \\ \frac{\partial y}{\partial v_2} &= 2v_2 \\ \frac{\partial v_1}{\partial x_1} &= 2x_1 \\ \frac{\partial v_2}{\partial x_1} &= x_2 \end{aligned}$$

Áp dụng quy tắc xích:

$$\begin{aligned} \frac{\partial y}{\partial x_1} &= \frac{\partial y}{\partial v_1}\frac{\partial v_1}{\partial x_1} + \frac{\partial y}{\partial v_2}\frac{\partial v_2}{\partial x_1} \\ &= 3(2x_1) + 2v_2(x_2) \end{aligned}$$

Thay $v_2 = x_1 x_2$ vào kết quả:

$$\frac{\partial y}{\partial x_1} = 6x_1 + 2x_1 x_2^2$$

4 Quy tắc xích dưới dạng tích của các ma trận Jacobi

Từ phát biểu tổng của tích, ta có thể tổng quát hóa quy tắc xích cho toàn bộ các vector đầu vào và đầu ra bằng cách sử dụng **Ma trận Jacobi (Jacobian matrix)**.

Cho hàm f là tổ hợp của hai hàm: $\mathbf{y} = h(\mathbf{v})$ và $\mathbf{v} = g(\mathbf{x})$. Trong đó $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{v} \in \mathbb{R}^k$, và $\mathbf{y} \in \mathbb{R}^m$. Ma trận Jacobi chứa tất cả các đạo hàm riêng bậc một của một hàm vector.

Quy tắc xích lúc này được viết gọn lại thành tích của hai ma trận Jacobi:

$$\mathbf{J}_{\mathbf{y},\mathbf{x}} = \mathbf{J}_{\mathbf{y},\mathbf{v}} \mathbf{J}_{\mathbf{v},\mathbf{x}} \tag{3}$$

Cấu trúc chi tiết của các ma trận:

- $\mathbf{J}_{\mathbf{v},\mathbf{x}}$ (kích thước $k \times n$): Biểu diễn sự thay đổi của các biến trung gian theo đầu vào.

$$\mathbf{J}_{\mathbf{v},\mathbf{x}} = \frac{\partial \mathbf{v}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial v_1}{\partial x_1} & \cdots & \frac{\partial v_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial v_k}{\partial x_1} & \cdots & \frac{\partial v_k}{\partial x_n} \end{pmatrix}$$

- $\mathbf{J}_{\mathbf{y},\mathbf{v}}$ (kích thước $m \times k$): Biểu diễn sự thay đổi của đầu ra theo các biến trung gian.

$$\mathbf{J}_{\mathbf{y},\mathbf{v}} = \frac{\partial \mathbf{y}}{\partial \mathbf{v}} = \begin{pmatrix} \frac{\partial y_1}{\partial v_1} & \cdots & \frac{\partial y_1}{\partial v_k} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial v_1} & \cdots & \frac{\partial y_m}{\partial v_k} \end{pmatrix}$$

Giải thích: Ma trận kết quả $\mathbf{J}_{\mathbf{y},\mathbf{x}}$ kích thước $m \times n$ thu được từ phép nhân ma trận chuẩn, bao hàm chính xác biểu thức tổng của các đạo hàm riêng dọc theo các đường dẫn tính toán.

Ví dụ minh họa

Bài toán: Xét vector đầu vào $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$. Cho các phép biến đổi: $\mathbf{v} = \begin{bmatrix} x_1^2 \\ x_1 x_2 \end{bmatrix}$ và $\mathbf{y} = \begin{bmatrix} v_1 + v_2 \\ 3v_2 \end{bmatrix}$. Tìm $\mathbf{J}_{\mathbf{y},\mathbf{x}}$.

Forward Pass:

$$v_1 = x_1^2$$
$$v_2 = x_1 x_2$$

$$y_1 = v_1 + v_2$$
$$y_2 = 3v_2$$

Backward Pass (Jacobians):

$$\mathbf{J}_{\mathbf{v},\mathbf{x}} = \begin{bmatrix} 2x_1 & 0 \\ x_2 & x_1 \end{bmatrix}$$

$$\mathbf{J}_{\mathbf{y},\mathbf{v}} = \begin{bmatrix} 1 & 1 \\ 0 & 3 \end{bmatrix}$$

Quy tắc xích:

$$\begin{aligned} \mathbf{J}_{\mathbf{y},\mathbf{x}} &= \mathbf{J}_{\mathbf{y},\mathbf{v}} \mathbf{J}_{\mathbf{v},\mathbf{x}} \\ &= \begin{bmatrix} 1 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 2x_1 & 0 \\ x_2 & x_1 \end{bmatrix} \\ &= \begin{bmatrix} 2x_1 + x_2 & x_1 \\ 3x_2 & 3x_1 \end{bmatrix} \end{aligned}$$

5 Quy tắc xích cho hàm vô hướng dưới dạng vector Gradient

Trong học máy, chúng ta thường xuyên làm việc với hàm mất mát (loss function) là một hàm vô hướng (scalar function). Tức là hàm có nhiều biến đầu vào nhưng chỉ trả về một giá trị duy nhất: $f : \mathbb{R}^n \to \mathbb{R}$ (với $m = 1$).

Đối với hàm vô hướng $y \in \mathbb{R}$, ma trận Jacobi của nó rút gọn thành một vector hàng. Biểu diễn dưới dạng vector cột, ta gọi nó là **Vector Gradient**, ký hiệu là ∇y. Nếu $y = h(\mathbf{v})$ và $\mathbf{v} = g(\mathbf{x})$, quy tắc xích liên kết vector gradient của đầu ra với ma trận Jacobi của lớp ẩn được biểu diễn như sau:

$$\nabla_{\mathbf{x}} y = \mathbf{J}_{\mathbf{v},\mathbf{x}}^T \nabla_{\mathbf{v}} y \tag{4}$$

Trong đó:

- $\nabla_{\mathbf{x}} y \in \mathbb{R}^{n \times 1}$ là vector gradient của y đối với các biến đầu vào $\mathbf{x}$.
- $\mathbf{J}_{\mathbf{v},\mathbf{x}}^T$ là chuyển vị của ma trận Jacobi của $\mathbf{v}$ theo $\mathbf{x}$, kích thước $n \times k$.
- $\nabla_{\mathbf{v}} y \in \mathbb{R}^{k \times 1}$ là vector gradient của y đối với các biến trung gian $\mathbf{v}$.

Đẳng thức này chính là cốt lõi của thuật toán lan truyền ngược, cho phép tính toán gradient bằng cách nhân ma trận chuyển vị từ lớp trên xuống lớp dưới.

Ví dụ minh họa

Bài toán minh họa: Lan truyền ngược tính Gradient của hàm mất mát qua một lớp ẩn.

Xét vector đầu vào $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$. Giả sử lớp ẩn thực hiện các phép biến đổi để tạo ra vector trung gian $\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} x_1^2 \\ x_1 x_2 \\ \sin(x_2) \end{bmatrix}$.

Hàm mất mát vô hướng ở đầu ra được định nghĩa là $y = v_1 + 2v_2 + v_3^2$. Dưới góc nhìn của Vi phân tự động, quá trình tính vector gradient $\nabla_{\mathbf{x}} y$ được chia thành các giai đoạn sau:

Forward Pass:

Tính toán các giá trị trung gian và lưu lại đồ thị:

$$v_1 = x_1^2$$
$$v_2 = x_1 x_2$$
$$v_3 = \sin(x_2)$$
$$y = v_1 + 2v_2 + v_3^2$$

Backward Pass:

Tính vector gradient (adjoints) của y theo $\mathbf{v}$:

$$\nabla_{\mathbf{v}} y = \begin{bmatrix} \frac{\partial y}{\partial v_1} \\ \frac{\partial y}{\partial v_2} \\ \frac{\partial y}{\partial v_3} \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 2v_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 2\sin(x_2) \end{bmatrix}$$

Tính ma trận Jacobi (đạo hàm cục bộ) của $\mathbf{v}$ theo $\mathbf{x}$:

$$\mathbf{J}_{\mathbf{v},\mathbf{x}} = \begin{bmatrix} \frac{\partial v_1}{\partial x_1} & \frac{\partial v_1}{\partial x_2} \\ \frac{\partial v_2}{\partial x_1} & \frac{\partial v_2}{\partial x_2} \\ \frac{\partial v_3}{\partial x_1} & \frac{\partial v_3}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 & 0 \\ x_2 & x_1 \\ 0 & \cos(x_2) \end{bmatrix}$$

Quy tắc xích :

Thuật toán lan truyền đạo hàm ngược về đầu vào bằng cách nhân chuyển vị của ma trận Jacobi với vector Gradient $\nabla_{\mathbf{v}} y$:

$$\begin{aligned} \nabla_{\mathbf{x}} y &= \mathbf{J}_{\mathbf{v},\mathbf{x}}^T \nabla_{\mathbf{v}} y \\ &= \begin{bmatrix} 2x_1 & x_2 & 0 \\ 0 & x_1 & \cos(x_2) \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 2\sin(x_2) \end{bmatrix} \\ &= \begin{bmatrix} (2x_1)(1) + (x_2)(2) + (0)(2\sin(x_2)) \\ (0)(1) + (x_1)(2) + (\cos(x_2))(2\sin(x_2)) \end{bmatrix} \\ &= \begin{bmatrix} 2x_1 + 2x_2 \\ 2x_1 + 2\sin(x_2)\cos(x_2) \end{bmatrix} \end{aligned}$$

Kiểm chứng tính nhất quán: Nếu thay trực tiếp biểu thức của $\mathbf{v}$ vào hàm mục tiêu, ta được $y = x_1^2 + 2x_1 x_2 + \sin^2(x_2)$. Việc lấy đạo hàm trực tiếp $\frac{\partial y}{\partial x_1}$ và $\frac{\partial y}{\partial x_2}$ cũng cho ra kết

quả hoàn toàn trùng khớp với thao tác nhân ma trận ở trên. Điều này minh họa rõ nét cơ chế hoạt động từng bước của thuật toán lan truyền ngược.

6 Cách vận hành và sự đúng đắn của Reverse M ode AD

Được ví như "người anh em họ"của thuật toán lan truyền ngược (backpropagation), **Reverse mode AD** hoạt động bằng cách lan truyền đạo hàm ngược từ một đầu ra cho trước. Kỹ thuật này thực hiện được điều đó bằng cách bổ sung cho mỗi biến trung gian v_i một **biến phụ hợp (adjoint)**, ký hiệu là $\bar{v}_i$:

$$\bar{v}_i = \frac{\partial y}{\partial v_i} \tag{5}$$

Biến $\bar{v}_i$ này đại diện cho *độ nhạy (sensitivity)* của đầu ra y đối với những thay đổi tại biến v_i.

Vận hành

Việc tính toán đạo hàm bằng Reverse mode về cơ bản là một quy trình hai bước:

1. **Forward Pass:** Mã nguồn của hàm gốc được chạy theo chiều thuận. Giai đoạn này khởi tạo các biến trung gian v_i và đặc biệt là phải theo dõi, lưu lại dấu vết của các mối quan hệ phụ thuộc trong đồ thị tính toán (computational graph).

2. **Reverse Sweep:** Các đạo hàm được tính toán bằng cách lan truyền các adjoint $\bar{v}_i$ theo chiều ngược lại, đi từ các đầu ra quay về các đầu vào.

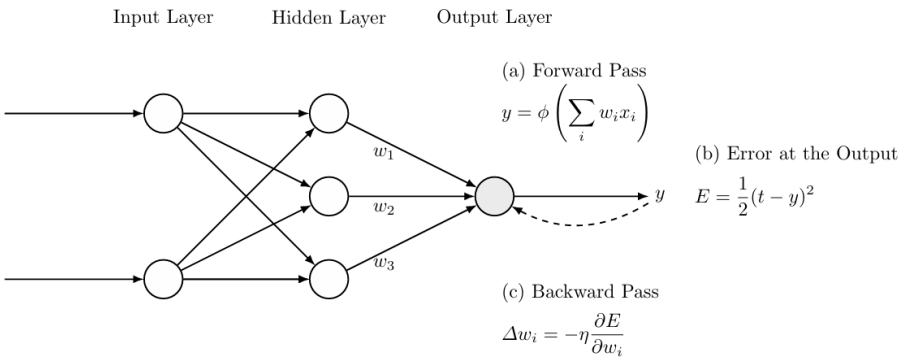
Về mặt toán : Xét trong đồ thị tính toán, giả sử một biến trung gian v_0 chỉ ảnh hưởng đến đầu ra y thông qua hai nhánh là biến v_2 và v_3. Sự đóng góp của v_0 vào sự thay đổi của y được biểu diễn bằng quy tắc xích:

$$\frac{\partial y}{\partial v_0} = \frac{\partial y}{\partial v_2}\frac{\partial v_2}{\partial v_0} + \frac{\partial y}{\partial v_3}\frac{\partial v_3}{\partial v_0} \tag{6}$$

Viết lại dưới dạng các biến phụ hợp (adjoints), ta có công thức lan truyền:

$$\bar{v}_0 = \bar{v}_2\frac{\partial v_2}{\partial v_0} + \bar{v}_3\frac{\partial v_3}{\partial v_0} \tag{7}$$

Sau khi hoàn thành lượt quét tiến, ta bắt đầu lượt quét lùi với giá trị khởi tạo tại đầu ra là $\bar{y} = \frac{\partial y}{\partial y} = 1$. Điểm mạnh cốt lõi của phương pháp này là: Đến cuối cùng, ta sẽ thu được toàn bộ đạo hàm của hàm số đối với các biến đầu vào (ví dụ: $\frac{\partial y}{\partial x_1} = \bar{x}_1$ và $\frac{\partial y}{\partial x_2} = \bar{x}_2$) **chỉ trong đúng một lượt quét lùi (one reverse sweep)**.



Hình 1: Hình minh họa thuật toán lan truyền ngược trong Neural Network (Nguồn: Automatic differentiation in machine learning: a survey)

Sự đúng đắn

Sự đúng đắn của Reverse mode AD đến từ việc áp dụng một cách chính xác quy tắc xích của giải tích dựa trên cấu trúc *sắp xếp tô-pô (topological sort)* của đồ thị tính toán có hướng. Bằng cách tính toán đạo hàm theo thứ tự lùi, khi hệ thống cần tính toán $\bar{v}_0$, tất cả các tác động của nó lên các biến phía sau (như $\bar{v}_2$ và $\bar{v}_3$) đều đã được tính toán hoàn tất trước đó. Điều này đảm bảo không có bất kỳ thao tác nào bị tính thiếu hoặc lặp lại, giữ nguyên độ chính xác tuyệt đối ở mức sai số máy (machine precision).

7 So sánh hai Mode của AD: Tại sao Reverse Mode vượt trội hơn trong Học Máy?

Sự khác biệt cốt lõi:

- **Forward Mode AD:** Đánh giá đạo hàm cùng lúc với việc tính toán hàm số từ đầu vào đến đầu ra. Nó lan truyền các giá trị *tiếp tuyến (tangents)* $\dot{v}_i = \frac{\partial v_i}{\partial x}$. Với một hàm $f : \mathbb{R}^n \to \mathbb{R}^m$, Forward Mode yêu cầu $\mathcal{O}(n)$ lượt duyệt (sweeps) để tính toán toàn bộ ma trận Jacobi.

- **Reverse Mode AD:** Tính toán hàm số theo chiều tiến, lưu lịch sử, rồi tính đạo hàm theo chiều lùi. Nó lan truyền các giá trị *phụ hợp (adjoints)* $\bar{v}_i = \frac{\partial y}{\partial v_i}$. Tương tự, nó yêu cầu $\mathcal{O}(m)$ lượt duyệt để hoàn thành toàn bộ ma trận Jacobi.

Chế độ nào vượt trội hơn?

Trong lĩnh vực Học máy (Machine Learning) và Deep Learning, **Reverse mode AD áp đảo hoàn toàn và là sự lựa chọn duy nhất có tính thực tiễn cao.**

Lý do: Bài toán phổ biến nhất trong ML là tối ưu hóa hàm mất mát vô hướng. Nghĩa là số lượng đầu ra $m = 1$, trong khi số lượng tham số (trọng số của mạng neural) n là cực kỳ lớn (thường lên tới hàng triệu hoặc hàng tỷ, $n \gg m$).

- Nếu dùng Forward Mode, ta sẽ phải thực hiện n lượt duyệt đồ thị tính toán (forward sweeps) - một con số không tưởng về mặt tài nguyên máy tính.
- Nhờ dùng Reverse Mode, do $m = 1$, ta chỉ cần **đúng MỘT lượt duyệt ngược duy nhất** là có thể thu được trọn vẹn vector Gradient cho toàn bộ hàng triệu tham số.

Bù lại, Reverse mode yêu cầu bộ nhớ (RAM/VRAM) lớn hơn đáng kể so với Forward mode vì nó phải lưu trữ lại toàn bộ "băng ghi"(tape) hay các biến trung gian của giai đoạn tiến để phục vụ cho giai đoạn lùi. Tuy nhiên, với tài nguyên phần cứng hiện nay, sự đánh đổi này là hoàn toàn xứng đáng và mang lại tốc độ vượt trội mang tính sống còn cho các mô hình AI.

Khởi tạo trọng số trong Perceptron nhiều lớp (MLP)

April 5, 2026

1 Giới thiệu

Mạng nơ-ron tiến (Feedforward Neural Networks) hay Perceptron nhiều lớp (MLP) học bằng cách điều chỉnh các trọng số (weights - W) và độ lệch (biases - b) để cực tiểu hóa hàm mất mát (Loss function) thông qua thuật toán tối ưu hóa Gradient Descent. Quá trình này bao gồm hai giai đoạn: lan truyền thuận (forward propagation) để tính toán đầu ra, và lan truyền ngược (backpropagation) để tính toán đạo hàm (gradient) nhằm cập nhật tham số.

Dù thuật toán lan truyền ngược rất mạnh mẽ, hiệu suất của nó bị ảnh hưởng nghiêm trọng bởi các điểm bắt đầu của quá trình tối ưu hóa. Nếu trọng số được khởi tạo không hợp lý, mạng có thể gặp phải hiện tượng triệt tiêu đạo hàm (vanishing gradient), bùng nổ đạo hàm (exploding gradient), hoặc mất khả năng phá vỡ tính đối xứng (symmetry breaking) [3].

2 Ví dụ về khởi tạo không tốt: Khởi tạo bằng 0 (Zero Initialization)

Một trong những cạm bẫy trực giác nhất là khởi tạo toàn bộ trọng số mạng nơ-ron bằng 0. Đối với các mô hình tuyến tính đơn giản như Hồi quy Logistic, khởi tạo bằng 0 vẫn hoạt động. Tuy nhiên, đối với MLP, đây là một thảm họa do vấn đề "tính đối xứng".

Xét một mạng MLP đơn giản có một lớp ẩn. Ký hiệu $x \in \mathbb{R}^d$ là đầu vào, $W^{(1)}$ là ma trận trọng số của lớp ẩn, và $f(\cdot)$ là hàm kích hoạt. Đầu ra của nơ-ron thứ i trong lớp ẩn được tính bằng:

$$a_i^{(1)} = f\left(z_i^{(1)}\right) = f\left(\sum_{j=1}^{d} W_{ij}^{(1)} x_j + b_i^{(1)}\right) \tag{1}$$

Nếu ta khởi tạo toàn bộ $W_{ij}^{(1)} = 0$ và $b_i^{(1)} = 0$, thì với mọi nơ-ron i, ta có $z_i^{(1)} = 0$. Dẫn đến:

$$a_i^{(1)} = f(0) \quad \forall i \tag{2}$$

Điều này có nghĩa là *tất cả các nơ-ron trong cùng một lớp ẩn đều tính toán ra cùng một giá trị đầu ra.*

Trong quá trình lan truyền ngược, đạo hàm của hàm mất mát L theo trọng số $W_{ij}^{(1)}$ được tính thông qua quy tắc chuỗi:

$$\frac{\partial L}{\partial W_{ij}^{(1)}} = \delta_i^{(1)} x_j \tag{3}$$

Trong đó $\delta_i^{(1)}$ là sai số (error term) của nơ-ron thứ i. Do cấu trúc mạng hoàn toàn đối xứng (các nơ-ron đóng góp như nhau vào dự đoán cuối cùng và có cùng đầu ra a_i), sai số truyền ngược về mỗi nơ-ron cũng sẽ hoàn toàn giống hệt nhau: $\delta_1^{(1)} = \delta_2^{(1)} = ... = \delta_n^{(1)}$.

3 Lập luận phương sai để giải thích khởi tạo Xavier

Việc khởi tạo ngẫu nhiên là cần thiết, nhưng ngẫu nhiên với độ lớn (phương sai) bao nhiêu lại là một bài toán khác. Năm 2010, Glorot và Bengio [1] đã đề xuất một phương pháp tiếp cận dựa trên lập luận bảo toàn phương sai (Variance argument), hay còn gọi là Xavier Initialization.

Mục tiêu của lập luận này là giữ cho luồng thông tin (phương sai của các activations) không bị khuếch đại (exploding) hay suy giảm về 0 (vanishing) khi đi qua hàng loạt các lớp của mạng sâu.

Xét một nơ-ron nhận n đầu vào x_i, với trọng số w_i. Tín hiệu tuyến tính z trước khi qua hàm kích hoạt là:

$$z = \sum_{i=1}^{n} w_i x_i \tag{5}$$

Để phân tích, ta đưa ra các giả định lý tưởng hóa:

- Các đầu vào x_i là các biến ngẫu nhiên độc lập, phân phối cùng nhau, có kỳ vọng $E[x_i] = 0$.
- Các trọng số w_i được khởi tạo độc lập với x_i và độc lập với nhau, có kỳ vọng $E[w_i] = 0$.

Sử dụng tính chất phương sai của biến độc lập, ta tính phương sai của đầu ra z:

$$Var(z) = Var\left(\sum_{i=1}^{n} w_i x_i\right) = \sum_{i=1}^{n} Var(w_i x_i) \tag{6}$$

Vì $E[x_i] = E[w_i] = 0$, phương sai của một tích phân rã thành:

$$Var(w_i x_i) = E[w_i^2 x_i^2] - (E[w_i x_i])^2 = E[w_i^2]E[x_i^2] - 0 = Var(w_i)Var(x_i) \tag{7}$$

Giả sử tất cả w_i có cùng phương sai $Var(w)$ và x_i có cùng phương sai $Var(x)$, ta có:

$$Var(z) = n \cdot Var(w) \cdot Var(x) \tag{8}$$

Để tín hiệu đầu vào không bị bóp méo khi truyền đi, ta muốn phương sai đầu ra bằng phương sai đầu vào, tức là $Var(z) = Var(x)$. Từ phương trình trên, ta ép điều kiện:

$$n \cdot Var(w) = 1 \implies Var(w) = \frac{1}{n} \tag{9}$$

Nếu xét cả chiều lan truyền ngược (phụ thuộc vào số nơ-ron lớp sau n_{out}), Glorot đề xuất lấy trung bình cộng để dung hòa cả hai hướng. Do đó, công thức khởi tạo Xavier phân phối các trọng số ngẫu nhiên sao cho:

$$Var(W) = \frac{2}{n_{in} + n_{out}} \tag{10}$$

4 So sánh hai cơ chế: Xavier (Glorot) và He (Kaiming)

Khởi tạo Xavier là một bước ngoặt lớn, nhưng nó được xây dựng dựa trên giả định rằng hàm kích hoạt là hàm tuyến tính, hoặc xấp xỉ tuyến tính quanh gốc tọa độ 0 (như hàm Tanh hay Sigmoid ở khoảng giá trị nhỏ).

Tuy nhiên, với sự trỗi dậy của hàm kích hoạt ReLU ($f(x) = \max(0, x)$), khởi tạo Xavier bắt đầu bộc lộ hạn chế. Do hàm ReLU "cắt bỏ"toàn bộ phần âm của phân phối, nó làm giảm phương sai của tín hiệu đầu ra đi một nửa so với giả định ban đầu.

Năm 2015, He et al. [2] đã đề xuất cơ chế khởi tạo He (Kaiming Initialization) để khắc phục điều này.

Sự khác biệt cốt lõi:

- **Phương sai khởi tạo:** Khởi tạo Xavier phân bổ trọng số với phương sai $Var(w) = \frac{1}{n_{in}}$ (hoặc $\frac{2}{n_{in}+n_{out}}$). Trong khi đó, để bù đắp sự mất mát phương sai do ReLU làm không hóa 50% tín hiệu, He Initialization nhân đôi phương sai mục tiêu lên: $Var(w) = \frac{2}{n_{in}}$.

- **Mô phỏng hàm kích hoạt:** Nếu dùng Xavier cho mạng dùng ReLU rất sâu (ví dụ 30 lớp), tín hiệu sẽ teo nhỏ dần đi một nửa qua mỗi lớp và biến mất hoàn toàn. Dùng khởi tạo He giúp bù đắp lượng tín hiệu hao hụt này, giữ cho phương sai duy trì ổn định ~ 1.0 xuyên suốt hàng chục lớp mạng.

Về mặt thực hành, đối với các kiến trúc mạng hiện đại, quy tắc kinh nghiệm là: 1. Nếu sử dụng **Sigmoid** hoặc **Tanh**, hãy sử dụng cơ chế **Xavier/Glorot**. 2. Nếu sử dụng **ReLU** hoặc các biến thể (Leaky ReLU), hãy sử dụng cơ chế **He/Kaiming**.

Tài liệu

[1] Xavier Glorot and Yoshua Bengio. *Understanding the difficulty of training deep feedforward neural networks*. In Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. *Delving deep into rectifiers: Surpassing human-level performance on imagenet classification*. In Proceedings of the IEEE international conference on computer vision, pages 1026–1034, 2015.

[3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. `http://www.deeplearningbook.org`

[4] Andrej Karpathy, Justin Johnson, Li Fei-Fei. *CS231n: Convolutional Neural Networks for Visual Recognition - Lecture Notes on Neural Networks Part 2*. Stanford University.